Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Архангельская Елизавета Павловна
Факультет прикладной информатики
Группа K3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025

Цель работы: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание 2.1.1:

1. *Создайте базу данных learn, заполните коллекцию единорогов unicorns*

```
test> use learn
switched to db learn
learn>
```

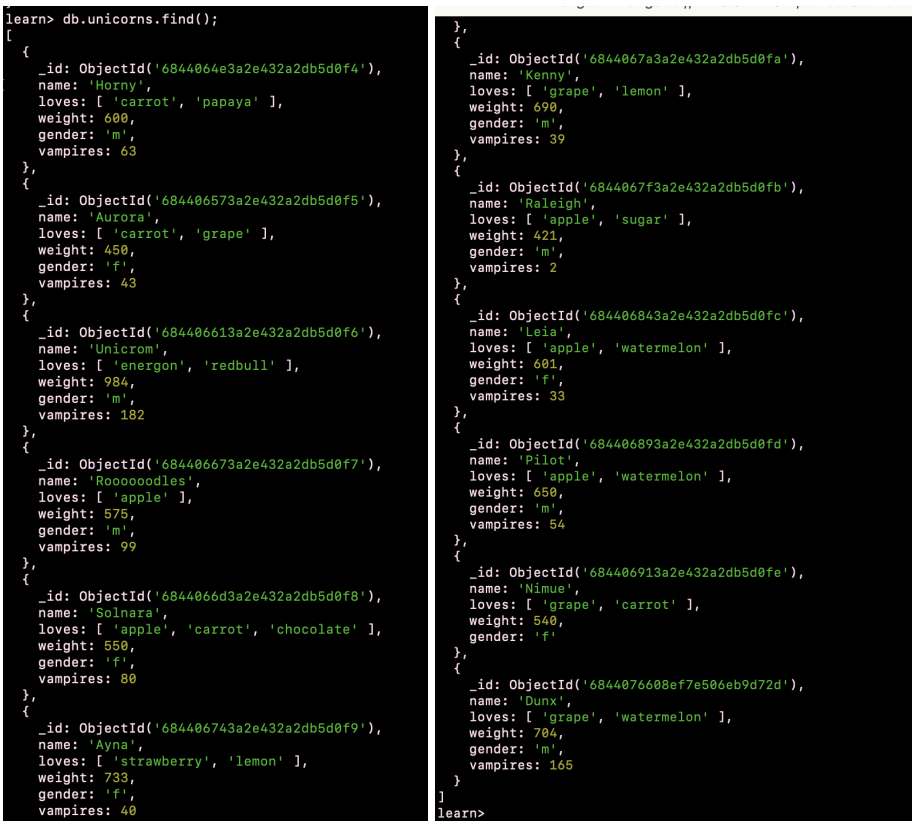
2. *Используя второй способ*

```
learn> db.unicorns.insertOne({name: 'Dunx', loves: ['grape', 'watermelon'], weig
ht: 704, gender: 'm', vampires: 165});
{
  acknowledged: true,
  insertedId: ObjectId('6844076608ef7e506eb9d72d')
}
learn>
```

3. *Вставьте в коллекцию единорогов документ*

db.unicorns.insertOne({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insertOne({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insertOne({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insertOne({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insertOne({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insertOne({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insertOne({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insertOne({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insertOne({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insertOne({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insertOne({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

4. *Проверьте содержимое коллекции с помощью метода find.*



```
learn> db.unicorns.find();
[
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('684406573a2e432a2db5d0f5'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('684406613a2e432a2db5d0f6'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('684406673a2e432a2db5d0f7'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6844066d3a2e432a2db5d0f8'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('684406743a2e432a2db5d0f9'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('684406843a2e432a2db5d0fc'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('684406913a2e432a2db5d0fe'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6844076608ef7e506eb9d72d'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn>
```

Практическое задание 2.2.1:

1. *Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.*



```
learn> db.unicorns.find({ gender: 'f' }).sort({ name: 1 }).limit(3);
[
  {
    _id: ObjectId('684406573a2e432a2db5d0f5'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('684406743a2e432a2db5d0f9'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('684406843a2e432a2db5d0fc'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

```
learn> db.unicorns.find({ gender: 'm'}).sort({ name: 1 }).limit(3);
[
  {
    _id: ObjectId('6844076608ef7e506eb9d72d'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

2. *Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.*

```
learn> db.unicorns.findOne({ gender: 'f', loves: 'carrot' });
{
  _id: ObjectId('684406573a2e432a2db5d0f5'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({ gender: 'f', loves: 'carrot' }).limit(1);
[
  {
    _id: ObjectId('684406573a2e432a2db5d0f5'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

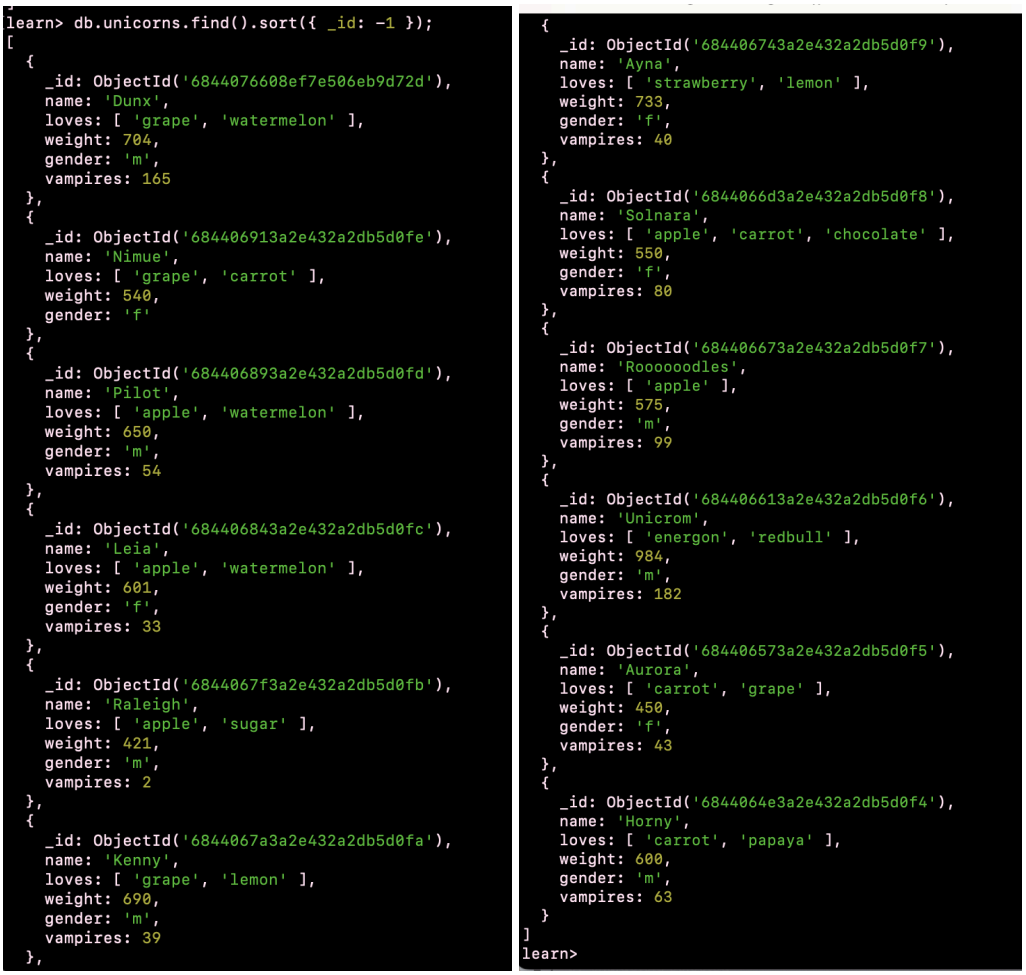
Практическое задание 2.2.2:

1. *Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.*

```
learn> db.unicorns.find({ gender: 'm' }, { loves: 0, gender: 0 }).sort({ name: 1 });
[
  {
    _id: ObjectId('6844076608ef7e506eb9d72d'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('684406673a2e432a2db5d0f7'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('684406613a2e432a2db5d0f6'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
```

Практическое задание 2.2.3:

1. *Вывести список единорогов в обратном порядке добавления.*



```
learn> db.unicorns.find().sort({ _id: -1 });
[
  {
    _id: ObjectId('6844076608ef7e506eb9d72d'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('684406913a2e432a2db5d0fe'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('684406843a2e432a2db5d0fc'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('684406743a2e432a2db5d0f9'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6844066d3a2e432a2db5d0f8'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('684406673a2e432a2db5d0f7'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('684406613a2e432a2db5d0f6'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('684406573a2e432a2db5d0f5'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
learn>
```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.aggregate([{$project: {_id: 0, name: 1, loves: { $arrayElemAt: ["$loves", 0] }}}]);
[
  { name: 'Horny', loves: 'carrot' },
  { name: 'Aurora', loves: 'carrot' },
  { name: 'Unicrom', loves: 'energon' },
  { name: 'Roooooodles', loves: 'apple' },
  { name: 'Solnara', loves: 'apple' },
  { name: 'Ayna', loves: 'strawberry' },
  { name: 'Kenny', loves: 'grape' },
  { name: 'Raleigh', loves: 'apple' },
  { name: 'Leia', loves: 'apple' },
  { name: 'Pilot', loves: 'apple' },
  { name: 'Nimue', loves: 'grape' },
  { name: 'Dunx', loves: 'grape' }
]
learn>
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 700 } }, { _id: 0 });
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: { $gte: 500 }, loves: { $all: ['grape', 'lemon'] }}, { _id: 0 });
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ `vampires`.

```
learn> db.unicorns.find({ vampires: { $exists: false } });
[
  {
    _id: ObjectId('684406913a2e432a2db5d0fe'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.aggregate([{ $match: { gender: 'm' } }, { $project: {_id: 0, name: 1, firstLove: { $arrayElemAt: ["$loves", 0] }}}, { $sort: { name: 1 } }]);
[
  { name: 'Dunx', firstLove: 'grape' },
  { name: 'Horny', firstLove: 'carrot' },
  { name: 'Kenny', firstLove: 'grape' },
  { name: 'Pilot', firstLove: 'apple' },
  { name: 'Raleigh', firstLove: 'apple' },
  { name: 'Roooooodles', firstLove: 'apple' },
  { name: 'Unicrom', firstLove: 'energon' }
]
learn>
```

Практическое задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
]
learn> db.towns.insertMany([{name: "Punxsutawney", populatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for: [""], mayor: {name: "Jim Wehrle" }},{name: "New Y
ork", populatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["status of liberty", "food"], mayor: {name: "Michael Bloomberg", party: "I"}}, {name: "Po
rtland", populatiuon: 528000, last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}}]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68440c6b08ef7e506eb9d72e'),
    '1': ObjectId('68440c6b08ef7e506eb9d72f'),
    '2': ObjectId('68440c6b08ef7e506eb9d730')
  }
}
learn>
```

2. *Сформировать запрос, который возвращает список городов с независимыми мэрами (*`party="I"`*). Вывести только название города и информацию о мэре.*

```
}
learn> db.towns.find({ "mayor.party": "I" }, { _id: 0, name: 1, mayor: 1 });
[
  {
    name: 'New York',
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn>
```

3. *Сформировать запрос, который возвращает список беспартийных мэров (*`party` *отсутствует*). *Вывести только название города и информацию о мэре.*

```
]
learn> db.towns.find({ "mayor.party": { $exists: false } },{ _id: 0, name: 1, mayor: 1 });
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Практическое задание 3.1.2:

1. *Сформировать функцию для вывода списка самцов единорогов.*

```
[learn> fn = function() { return this.gender == 'm'; }
[Function: fn]
[learn> db.unicorns.find({ $where: fn });
[
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('684406613a2e432a2db5d0f6'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('684406673a2e432a2db5d0f7'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
```

2. *Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.*

```
var cursor = db.unicorns.find({ $where: fn }).sort({ name: 1 }).limit(2);
```

3. *Вывести результат, используя forEach.*

```
]
[learn> var cursor = db.unicorns.find({ $where: fn }).sort({ name: 1 }).limit(2);

[learn> cursor.forEach(function(obj) {print(obj.name);});
Dunx
Horny

learn>
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.countDocuments({ gender: 'f', weight: { $gte: 500, $lte: 600 } });
2
learn>
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves");
[
  'apple',      'carrot',
  'chocolate',  'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: { $sum: 1 }}}]);
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn>
```

Практическое задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barny', loves: ['grape'],
weight: 340, gender: 'm'})
```

```
learn> db.unicorns.save({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'});
TypeError: db.unicorns.save is not a function
learn>
```

Это произошло потому что метод `save()` больше не поддерживается в новых версиях MongoDB. Вместо него использую `insertOne()`:

```
learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'});
{
  acknowledged: true,
  insertedId: ObjectId('68441217aee5b7bf3b6ac079')
}
learn>
```

2. *Проверить содержимое коллекции* `unicorns`.

Добавился один элемент.



Практическое задание 3.3.2:

1. *Для самки единорога* `Ayna` *внести изменения в БД: теперь ее вес 800, она убила 51 вапмира.*



2. *Проверить содержимое коллекции* `unicorns`.



Обновился один элемент.

Практическое задание 3.3.3:

1. *Для самца единорога* `Raleigh` *внести изменения в БД: теперь он любит рэдбул.*

```
learn> db.unicorns.updateOne({ name: "Raleigh", gender: "m" }, { $addToSet: { loves: "Red Bull" } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

2. *Проверить содержимое коллекции* `unicorns`.

```
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'Red Bull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
```

Практическое задание 3.3.4:

1. *Всем самцам единорогов увеличить количество убитых вапмиров на 5*

```
learn> db.unicorns.updateMany({ gender: "m" }, { $inc: { vampires: 5 } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn>
```

2. *Проверить содержимое коллекции* `unicorns`.

```
  }
]
learn> db.unicorns.find({gender: "m"});
[
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('684406613a2e432a2db5d0f6'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('684406673a2e432a2db5d0f7'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6844067a3a2e432a2db5d0fa'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44
  },
  {
    _id: ObjectId('6844067f3a2e432a2db5d0fb'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'Red Bull' ],
    weight: 421,
    gender: 'm',
    vampires: 7
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  },
  {
    _id: ObjectId('6844076608ef7e506eb9d72d'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 170
  },
  {
    _id: ObjectId('68441217aee5b7bf3b6ac079'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm',
    vampires: 5
  }
]
learn>
```

Практическое задание 3.3.5:

1. *Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.*

```
[]
learn> db.towns.updateOne({ name: "Portland" }, { $unset: { "mayor.party": "" } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

2. *Проверить содержимое коллекции* `towns`.

```
learn> db.towns.find();
[
  {
    _id: ObjectId('68440c6b08ef7e506eb9d72e'),
    name: 'Punxsutawney',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('68440c6b08ef7e506eb9d72f'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('68440c6b08ef7e506eb9d730'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>
```

Практическое задание 3.3.6:

1. *Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.*

```
learn> db.unicorns.updateOne({ name: "Pilot", gender: "m" }, { $addToSet: { loves: "chocolate" } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

2. *Проверить содержимое коллекции* `unicorns`.

```
  },
  {
    _id: ObjectId('684406893a2e432a2db5d0fd'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  },
```

Практическое задание 3.3.7:

1. *Изменить информацию о самке единорога* `Aurora`: *теперь она любит еще и сахар, и лимоны.*

```
learn> db.unicorns.updateOne({ name: "Aurora", gender: "f" }, { $addToSet: { loves: { $each: ["sugar", "lemon"] } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

2. *Проверить содержимое коллекции* `unicorns`.

```
  },
  {
    _id: ObjectId('684406573a2e432a2db5d0f5'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
```

Практическое задание 3.4.1:

1. *Удалите документы с беспартийными мэрами.*

```
learn> db.towns.deleteMany({ "mayor.party": { $exists: false } });
{ acknowledged: true, deletedCount: 2 }
learn>
```

2. *Проверьте содержание коллекции.*

```
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find();
[
  {
    _id: ObjectId('68440c6b08ef7e506eb9d72f'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn>
```

3. *Очистите коллекцию.*

```
]
learn> db.towns.deleteMany({});
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find();

learn>
```

4. *Просмотрите список доступных коллекций.*

```
learn> show collections
towns
unicorns
learn>
```

Практическое задание 4.1.1:

1. *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*

```
learn> db.habitats.insertMany([{ _id: "MOW", fullName: "Moscow", description: "Capital of Russia" },{ _id: "SPB", fullName: "Sa
int-Petersburg", description: "Cultural capital of Russia" }, { _id: "LDN", fullName: "London", description: "Is the capital of
 Great Britain" }, { _id: "NYC", fullName: "New York", description: "Largest city in the USA" }]);
{
  acknowledged: true,
  insertedIds: { '0': 'MOW', '1': 'SPB', '2': 'LDN', '3': 'NYC' }
}
learn>
```

2. *Включите для нескольких единорогов в документы ссылку на зону обитания, использую второй способ автоматического связывания.*

```
learn> db.unicorns.updateOne({ name: "Horny" },{ $set: { habitat: { $ref: "habitats", $id: "MOW" } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({ name: "Ayna" },{ $set: { habitat: { $ref: "habitats", $id: "LDN" } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

3. *Проверьте содержание коллекции единорогов.*

```
  {
    _id: ObjectId('684406743a2e432a2db5d0f9'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51,
    habitat: DBRef('habitats', 'LDN')
  },
```

```
[
  {
    _id: ObjectId('6844064e3a2e432a2db5d0f4'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    habitat: DBRef('habitats', 'MOW')
  },
```

Практическое задание 4.2.1:

1. *Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.*

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true });
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

Можно.

Практическое задание 4.3.1:

1. *Получите информацию о всех индексах коллекции* `unicorns`.
2. *Удалите все индексы, кроме индекса для идентификатора.*
3. *Попытайтесь удалить индекс для идентификатора.*

```
learn> db.unicorns.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes();
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.dropIndex("_id")
MongoServerError[IndexNotFound]: index not found with name [_id]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Практическое задание 4.4.1:

1. *Создайте объемную коллекцию* `numbers`*, задействовав курсор:*

   `for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}`

```
learn> for(let i = 0; i < 100000; i++) { db.numbers.insertOne({ value: i }) };
{
  acknowledged: true,
  insertedId: ObjectId('68441c1214b9754c9da9e196')
}
learn>
```

2. *Выберите последних четыре документа.*

```
learn> db.numbers.find().sort({ _id: -1 }).limit(4);
[
  { _id: ObjectId('68441c1214b9754c9da9e196'), value: 99999 },
  { _id: ObjectId('68441c1214b9754c9da9e195'), value: 99998 },
  { _id: ObjectId('68441c1214b9754c9da9e194'), value: 99997 },
  { _id: ObjectId('68441c1214b9754c9da9e193'), value: 99996 }
]
learn>
```

3. *Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра executionTimeMillis)*

```
learn> db.numbers.find().sort({ _id: -1 }).limit(4).explain("executionStats");
```

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
```

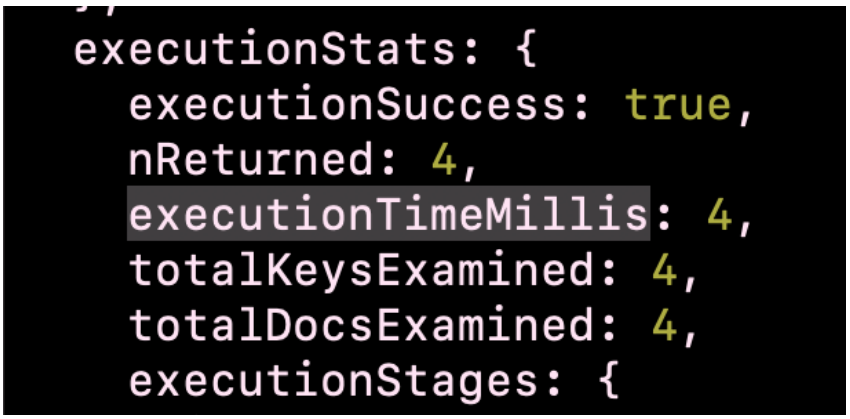
4. *Создайте индекс для ключа value.*
5. *Получите информацию о всех индексах коллекции numbres.*

```
learn> db.numbers.createIndex({ value: 1 });
value_1
learn> db.numbers.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>
```

6. *Выполните запрос 2.*

```
learn> db.numbers.find().sort({_id: -1 }).limit(4);
[
  { _id: ObjectId('68441c1214b9754c9da9e196'), value: 99999 },
  { _id: ObjectId('68441c1214b9754c9da9e195'), value: 99998 },
  { _id: ObjectId('68441c1214b9754c9da9e194'), value: 99997 },
  { _id: ObjectId('68441c1214b9754c9da9e193'), value: 99996 }
]
learn> db.numbers.find().sort({value: -1 }).limit(4).explain("executionStats");
{
```

7. *Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?*



8. *Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?*

Более эффективный запрос с _id, а не с value, т.к. поле _id в MongoDB автоматически индексируется. Сортировка и поиск по _id происходит очень быстро, поскольку данные уже упорядочены по этому полю, а индекс по value создан отдельно. При сортировке по value MongoDB использует именно этот индекс, он не так оптимален по сравнению с _id.

Вывод:

В ходе лабораторной работы были освоены основные операции в MongoDB: создание баз, коллекций, вставка, обновление, удаление и выборка данных. Также были изучены индексы, связи между коллекциями и анализ производительности запросов с помощью explain. Было установлено, что запросы по полю _id выполняются быстрее за счёт встроенного индекса. Работа позволила закрепить навыки эффективной работы с MongoDB и оптимизации запросов.